

Break the login

Raport scris

Nichita Iulia-Nicoleta, 341

Aprilie 2026

1 Introducere

Compania fictivă *AuthX* utilizează o aplicație web pentru a gestiona accesul la resurse sensibile al angajaților săi. Pentru a îndeplini această condiție, aplicația salvează datele utilizatorilor (Users), biletele stocate (Tickets) și monitorizează activitățile (Audit logs).

Un utilizator este recunoscut din momentul înregistrării în platformă prin *formularul de register*, iar apoi cu aceste date se poate loga prin *formularul de login*. Utilizatorii sunt înregistrați cu rolul fiecăruia, ANALYST sau MANAGER. Rolul de ANALYST permite vizualizarea și editarea propriilor bilete, precum și editarea profilului. Rolul de MANAGER are aceleași permisiuni ca cele de ANALYST la care se adaugă și vizualizarea activităților înregistrate. Indiferent de rol, utilizatorilor le este permis să își reseteze parola contului personal prin *formularul de resetpassword*, adăugând email-ul contului și adăugând noua parolă pe o pagină cu acest scop.

Arhitectură

Aplicația AuthX utilizează o arhitectură client-server clasică, în care frontend-ul comunică cu backend-ul printr-un API REST. Frontend-ul aplicației este compus din pagini *HTML statice*, stilizate cu CSS, care utilizează formulare pentru a primi datele introduse de utilizatori în câmpurile dedicate. Schimbul de date dintre interfață și server se realizează prin logica de fetch, utilizând formatul JSON și metodele standard HTTP: GET, POST, PUT și DELETE. S-a ales HTML static pentru simplitate și timp de încărcare redus, fără overhead de framework.

Backend-ul este implementat prin framework-ul *Express*. Are rolul de a procesa cererile primite, de a realiza logica funcționalităților platformei și de a gestiona accesul la baza de date prin metodele enumerate anterior. Acesta expune un set de endpoint-uri REST prin care frontend-ul poate efectua operații asupra entităților aplicației: users, tickets, audit_logs. Baza de date aleasă este *SQLite*, datorită simplității, portabilității și performanței, fiind o

bază de date relațională ce permite utilizarea metodelor de tip CRUD și o soluție de tip *zero-configuration*," stocată într-un singur fișier, fără a necesita un server dedicat.

Aplicația adoptă practici de securitate cibernetică pentru protecția împotriva XSS(Cross-Site Scripting), CSRF(Cross-Site Request Forgery), pentru prevenirea injection sau hijacking. Au fost alese în acest sens: JWT (JSON Web Token implementat prin librăria `jsonwebtoken`) trimis printr-un cookie de sesiune, bcrypt, crypto (build-in Node.js module).

2 Setup mediu

Această secțiune descrie pașii necesari pentru pregătirea mediului de lucru și instalarea dependențelor software.

2.1 Virtualizare și mediu de lucru

Pentru dezvoltarea aplicației s-au utilizat:

- Sistem de operare folosit: Windows 11
- Node.js: v24.11.0
- npm: 11.6.1
- Bază de date utilizată: SQLite (sqlite3 library - 3.46.1 (64-bit))
- Editor: VS Code

Utilizarea unei mașini virtuale (VirtualMachine) este justificată prin beneficiile tehnice aduse aplicației AuthX. S-a utilizat Oracle VirtualBox (Version 7.1.2) pentru protejarea sistemului gazdă (host) de eventuale vulnerabilități testate în timpul dezvoltării, asigurarea că aplicația rulează pe o configurație „curată”, fără conflicte cu alte programe instalate pe calculatorul personal.

Configurația Hardware VirtualMachine:

- Procesor (vCPU): 5 nuclee
- Memorie RAM: 16384 MB
- Stocare: 50GB (dinamic alocată)
- Rețea: NAT (pentru acces la internet)

2.2 Instalare și utilizare a framework-ului și a bazei de date

Express este un framework pentru dezvoltarea aplicațiilor web ce utilizează *Node.js* și oferă acces la funcționalități esențiale, intuitiv create, ușor de utilizat pentru a ajuta dezvoltatorii să creeze aplicații ce demonstrează scalabilitate și performanță ridicată.

Configurație 1: Instalare Node.js și Express

```
# Instalare Node.js
sudo apt update
sudo apt install nodejs npm      # Linux

# Verificare versiuni
node -v
npm -v

# Creeaza package.json*
npm init

npm install express
```

*package.json este creat pentru identificarea proiectului și gestionarea dependențelor

Pentru a asigura o structură modulară, proiectul valorifică funcționalități ale framework-ului Express, facilitând astfel implementarea unui setup minimalist cu metodele disponibile (ex: `express()`, `app.listen()`), Express fiind considerat un *Minimalist framework*.

Acest framework pune la dispoziție un *sistem de rutare* ce permite definirea rutelor pentru metodele HTTP cunoscute (GET, POST, PUT, DELETE), ceea ce aduce flexibilitate aplicației dezvoltate.

Cod Sursă 1: Definire rute, punerea în funcțiune a server-ului

```
import express from 'express';

const app = express();
const port = 3000;

app.use(express.json()); /*

app.get('/login', (req, res) => {
  res.sendFile(path.join(__dirname, './templates/login.html'));
});
// login.html - HTML static ce afiseaza formularul de login

// ...
```

```
app.listen(port, () => {
  console.log('App listening at http://localhost:${port}');
});
```

* `express.json()` definește un middleware definit de Express ce permite server-ului să par-seze cereri cu payload în format JSON.

Deoarece Express utilizează mediu de execuție Node.js, aplicația poate gestiona *cereri asincrone*. Acest aspect permite server-ului să proceseze mai multe cereri simultan, ceea ce îmbunătățește performanța sistemului.

Gestionarea răspunsurilor la cererile primite este un proces important în aplicații web și implicit *tratarea erorilor*. Platforma utilizează o abordare de tratare a erorilor pe mai multe niveluri. Strategia se concentrează pe prevenirea stărilor de "crash" ale serverului și pe furnizarea de coduri de stare corespunzătoare.

Înainte de interacțiunea cu baza de date, aplicația verifică integritatea datelor primite în corpul cererii (`req.body`). Pentru a asigura consistența și a evita duplicarea codului, a fost implementată funcția helper `badRequest` (vezi în *Cod sursă 2* și *Cod sursă 3*), care returnează un cod de stare **400 Bad Request**. Interacțiunile cu baza de date SQLite sunt asincrone și utilizează modelul error-first callback (Vezi *Cod sursă 2*). Deoarece biblioteca `jsonwebtoken` aruncă excepții în mod sincron atunci când un token este invalid sau expirat, procesul de verificare (`jwt.verify`) este încapsulat într-un bloc `try-catch` (vezi în *Cod sursă 4*). Acest lucru previne oprirea serverului și permite returnarea unui răspuns de tip **401 Unauthorized**.

Cod Sursă 2: Cerere de tip POST, tratare erori (fișier `'/api/user'`)

```
import express from 'express';

const router = express.Router();

// *
function badRequest(res, message) {
  return res.status(400).json({ error: message });
}

// creare user nou
// cerere de tip POST
router.post('/register', (req, res) => {
  const { email, password, role, locked } = req.body;

  // ... (logica validare input)

  const sql = // ... (interogare catre baza de date SQLite)
```

```

    // rularea interogarii pentru salvarea user-ului
    db.run(sql, [email, password, role, normalizedLocked],
function (err) {
    // tratare erori**
    // error-first callback
    if (err) return res.status(500).json({ error: err.message
});

    // ... (logica salvare actiune in audit_logs)

    // 201 successful HTTP request
    res.status(201).json({
        id: this.lastID,
        email,
        role,
        // ...
    });
});
});

export default router;

```

*în cazul datelor neconforme introduse de utilizator la completarea formularului, este returnat un răspuns cu statutul 400, considerat Client Error (datele introduse de client nu corespund)

erorile survenite în timpul interogării bazei de date sunt tratate local, returnând imediat un răspuns cu statusul **500 Internal Server Error, iar codul verifică prezența obiectului de eroare înainte de a procesa rezultatele, asigurând astfel integritatea fluxului de date

Cod Sursă 3: Definirea rutei pentru gestionarea cererilor API

```
app.use('/api/user', userRoutes);
```

Cod Sursă 4: Cerere de tip PUT, tratare erori în bloc `try-catch`

```

// Update current user profile
// cerere de tip PUT
router.put('/profile', (req, res) => {
    const token = req.cookies.auth_token;

    if (!token) return res.status(401).json({ error: 'Login
required' });

```

```

let decoded;
try {
  // extragerea payload-ului
  decoded = jwt.verify(token, 'abc');
} catch (err) {
  // 401 Unauthorized
  return res.status(401).json({ error: 'Invalid token' });
}

const { email } = req.body;

if (!email) {
  return badRequest(res, 'email cannot be empty');
}

// rulare interogare baza de date
db.run('UPDATE users SET email = ? WHERE id = ?', [email,
decoded.userId], function (err) {
  if (err) return res.status(500).json({ error: err.message
});
  if (this.changes === 0) return res.status().json({ error:
'User not found' });

  // ... (logica salvare actiune in audit_logs)

  res.json({ updatedID: decoded.userId });
});
});

```

Aplicația AuthX returnează și alte coduri de stare, precum **403 forbidden**. Unul dintre contextele în care este declanșată este încercarea unui user cu rol ANALYST de a șterge un cont care nu este al său (Vezi *Cod sursă 5*). Aplicația returnează și **404 Not found**, declanșată la apelul endpoint-ului `/api/user/login` la trimiterea unui input de email care nu există (`error: 'User not found'`) (Vezi *Cod sursă 6*).

Cod Sursă 5: Cerere de tip PUT, tratare erori în bloc `try-catch`

```

// Delete user
// cerere de tip PUT
router.delete('/:id', (req, res) => {
  const userId = Number(req.params.id);

  const token = req.cookies.auth_token;

```

```

    if (!token) return res.status(401).json({ error: 'Login
required' });

    try {
        const decoded = jwt.verify(token, 'abc');

        if (decoded.userId !== userId) {
            // 403 Forbidden
            return res.status(403).json({ error: 'You can only
delete your own account' });
        }

        db.run('DELETE FROM users WHERE id = ?', [userId],
function (err) {
            if (err) return res.status(500).json({ error: err.
message });
            if (this.changes === 0) return res.status(404).json({
error: 'User not found' });

            // ... (logica salvare actiune in audit_logs)

            res.json({ deletedID: userId });
        });
    } catch (err) {
        res.status(401).json({ error: 'Invalid token' });
    }
});

```

Cod Sursă 6: Cerere de tip POST, implementare logică login

```

// Login
// cerere de tip POST
router.post('/login', (req, res) => {
    const { email, password_hash } = req.body;

    // interogare catre baza de date pentru determinarea user-ului
    db.get('SELECT * FROM users WHERE email = ?', [email], async (
err, user) => {
        if (err) return res.status(500).json({ error: err.message
});
        if (!user) {
            // ... (logica salvare actiune in audit_logs)

            // 404 Not found
            return res.status(404).json({ error: 'User not found'
});
        }
    });
});

```

```

});
    }

    // ... (logica validare parola)

    // parola este corecta
    if (match) {
        // ... logica creeare token

        // ... (logica salvare token in cookie)

        // ... (logica salvare actiune in audit_logs)

        res.json({ success: true, message: "Authentication
successful" });
    } else {
        // ... (logica salvare actiune in audit_logs)

        res.status(401).json({ error: 'Incorrect password' });
    }
});
});

```

Pentru gestionarea persistentei datelor, proiectul utilizează **SQLite**, un motor de bază de date relațional de tip embedded. Integrarea în mediul Node.js a fost realizată prin intermediul bibliotecii `sqlite3`, utilizând interogări parametrizate pentru a preveni atacurile de tip SQL Injection. Pachetul `sqlite3` oferă driverul necesar pentru ca codul Javascript să poată comunica cu fișierul bazei de date.

Configurație 2: Instalare librărie

```
npm install sqlite3
```

Următorul pas este crearea unei conexiuni către fișierul SQLite (`'data.sqlite'`). Dacă acesta nu există, SQLite îl va crea automat. Acest fișier este de fapt un sistem de fișiere structurat extrem de complex ce conține toată baza de date definită.

Pentru aplicația AuthX, baza de date stochează datele în 3 entități: users, tickets, audit_logs.

Entitatea users are următoarele câmpuri: id, email, password_hash, role, reset_token, reset_token_expired_at, created_at, locked. Sunt stocate date despre conturile utilizato-

rilor, `reset_token`, precum și `reset_token_expired_at` sunt utilizate la funcționalitatea de resetare a parolei contului personal, iar `locked` este utilizat pentru a identifica un cont blocat, din cauza încercărilor multiple de logare (credential stuffing / brute-force).

Entitatea `tickets` are următoarele câmpuri: `id`, `title`, `description`, `severity`, `status`, `owner_id`, `created_at`, `updated_at`. Sunt stocate edate despre biletele existente în aplicație, iar câmpul `owner_id` este cheie străină către entitatea `users`, câmpul `users.id` pentru a identifica cine este deținătorul biletului.

Entitatea `audit_logs` are următoarele câmpuri: `id`, `user_id`, `action`, `resource`, `resource_id`, `timestamp`, `ip_address`. Sunt stocate date despre activitățile din platformă, precum: crearea unui cont nou, logarea într-un cont existent, resetarea parolei, adăugarea unui nou bilet (de către manager). Câmpul `user_id` este cheie străină către entitatea `users`, câmpul `users.id` pentru a determina ce user a realizat acțiunea, câmpul `resource` poate fi `USERS` sau `TICKETS` pentru a detemrina asupra cui a avut loc acțiunea, iar câmpul `resource_id` este utilizat pentru a reține id-ul resursei asupra căreia s-a realizat acțiunea.

Cod Sursă 7: Creare conexiune către baza de date și crearea tabelor

```
import sqlite3 from 'sqlite3';

const dbPath = './data.sqlite';
const db = new sqlite3.Database(dbPath, (err) => {
  if (err) console.error(err.message);

  console.log('Connected to SQLite.');
```

});

db.serialize(() => {
 // fara aceasta comanda se pot introduce id-uri care nu exista
 db.run('PRAGMA foreign_keys = ON', (err) => {
 if (err) console.error("Failed to enable FK support:", err
.message);
 });

 db.run('CREATE TABLE IF NOT EXISTS users (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 email TEXT UNIQUE NOT NULL,
 password_hash TEXT NOT NULL,
 role TEXT CHECK(role IN ('ANALYST', 'MANAGER')) NOT NULL,
 created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
 reset_token TEXT,
 reset_token_expires_at DATETIME,
 locked INTEGER DEFAULT 00 CHECK(locked IN (0, 1))
)', (err) => { if (err) console.error('users:', err.message);
});

 db.run('CREATE TABLE IF NOT EXISTS tickets (

```

        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        description TEXT,
        severity TEXT CHECK(severity IN ('LOW', 'MED', 'HIGH'))
NOT NULL,
        status TEXT CHECK(status IN ('OPEN', 'IN_PROGRESS', '
RESOLVED')) NOT NULL,
        owner_id INTEGER,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (owner_id) REFERENCES users(id)
            ON DELETE CASCADE
            ON UPDATE CASCADE
    ), (err) => { if (err) console.error('tickets:', err.message)
; });

db.run('CREATE TABLE IF NOT EXISTS audit_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    action TEXT NOT NULL,
    resource TEXT,
    resource_id TEXT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    ip_address TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    ), (err) => { if (err) console.error('audit_logs:', err.
message); });

});

export default db;

```

SQLite folosește metode diferite în funcție de tipul de rezultat așteptat:

- `db.run()`: Pentru acțiuni care nu returnează date (INSERT, UPDATE, DELETE).
- `db.get()`: Pentru a prelua un singur rând (ex: la Login).
- `db.all()`: Pentru a prelua toate rândurile dintr-o listă (ex: /assignees).

Cod Sursă 7: Interogări către baza de date SQLite

```
// .GET
```

```

db.get('SELECT * FROM users WHERE email = ?', [email], async (err,
  user) => {
    if (err) return res.status(500).json({ error: err.message
    });
    if (!user) {
      // ... (logica salvare actiune in audit_logs) (Vezi
      Cod sursa 8)

      return res.status(404).json({ error: 'User not found'
    });
  }

  // ... (logica validare parola)

  // daca parola este corecta
  if (match) {
    // ... (logica login cu succes)
  }
});

// .RUN
db.run('UPDATE users SET email = ? WHERE id = ?', [email, decoded.
  userId], function (err) {
  if (err) return res.status(500).json({ error: err.message });
  if (this.changes === 0) return res.status(404).json({ error: '
  User not found' });

  // ... (logica salvare actiune in audit_logs) (Vezi Cod sursa
  8)

  res.json({ updatedID: decoded.userId });
});

// .ALL
db.all('SELECT id, email, role FROM users ORDER BY email ASC', [],
  (err, users) => {
    if (err) return res.status(500).json({ error: err.message });

    // ... (logica returnare raspuns)
  });

```

Pentru salvarea acțiunilor în tabelul audit_logs au fost create 2 funcții suplimentare pentru determinarea adresei ip a browser-ului care a făcut cererea, precum și salvarea în baza de date, popularea câmpurilor înregistrării cu date corespunzătoare.

Cod Sursă 8: Funcții suplimentare pentru noi înregistrări în entitatea audit_logs

```
import db from '../database.js';

function getClientIp(req) {
  // 'x-forwarded-for' contine adresa ip a cererii
  const forwarded = req.headers['x-forwarded-for'];

  if (typeof forwarded === 'string' && forwarded.trim() !== '')
  {
    return forwarded.split(',')[0].trim();
  }

  const ip = req.ip ?? null;

  // converteste ::1 la 127.0.0.1
  if (ip === '::1') return '127.0.0.1';

  // converteste ::ffff:192.168.1.1 la 192.168.1.1
  if (ip?.startsWith('::ffff:')) return ip.substring(7);

  return ip;
}

export function logAudit({ req, userId = null, action, resource =
  null, resourceId = null }) {
  if (!action) {
    return;
  }

  const resourceIdValue = resourceId === null || resourceId ===
    undefined
    ? null
    : String(resourceId);

  // inserare in baza de date
  db.run(
    `
      INSERT INTO audit_logs (user_id, action, resource,
        resource_id, ip_address)
      VALUES (?, ?, ?, ?, ?)
    `,
    [userId, action, resource, resourceIdValue, getClientIp(
      req)],
    (err) => {
      if (err) {
        console.error('Audit log insert failed:', err.
          message);
      }
    }
  );
}
```

```

    }
  }
);
}

```

Pentru vizualizarea mai ușoară a entităților bazei de date este utilizat DB Browser for SQLite Version 3.13.1.

SQLite nu suportă tipul de date BOOLEAN nativ (folosește 0 și 1), de aceea există o funcție `normalizeLocked` în cod.

Cod Sursă 9: Funcție suplimentară pentru tipul de date BOOLEAN

```

function normalizeLocked(value) {
  if (value === undefined) return undefined;
  if (value === true || value === 1 || value === '1') return 1;
  if (value === false || value === 0 || value === '0') return 0;
  return null;
}

```

3 Implementare MVP

Sistem de autentificare

Aplicația AuthX implementează un sistem de autentificare ce include logica de înregistrare în platformă (register), precum și logica de logare într-un cont existent (login). Sistemul este construit pe standardul **JWT** (JSON Web Token) și utilizează mecanisme de stocare securizată prin **HTTP Cookies**, precum și librăria `bcrypt` pentru hashuirea parolelor stocate și modulul `crypto` pentru a crea un token aleator în contextul resetării parolei. Procesul de autentificare urmează o logică strictă pentru a preveni accesul neautorizat:

1. Register:

- utilizatorul completează un formular de creare cont cu email, password și role (ANALYST sau MANAGER)
- datele de input sunt validate în backend, la nivelul cererii `/api/user/register`
- dacă toate datele sunt valide (ex: password de o lungime mai mare de 8 caractere ce conține cel puțin o literă mare, o literă mică, un număr și un caracter special, etc.) sunt stocate sub forma unei înregistrări în tabelul `users`, cu input-ului din câmpul password hash-uit (`bcrypt.hash`)

- dacă un câmp nu trece de pasul de validare, utilizatorul primește un mesaj în dreptul acelui câmp cu requirement-ul necesar

2. Login:

- utilizatorul completează un formular cu email și password
- se verifică dacă există un utilizator cu email-ul respectiv în baza de date
- dacă există, se hash-uieste parola (password) pentru a fi comparată cu cea stocată în baza de date(`bcrypt.compare`)
 - dacă parolele corespund se creează un token (`jwt.sign`) ce are ca payload date despre utilizatorul logat (userId, și dacă este manager pentru verificării identității) și este salvat într-un cookie de sesiune și transmis la cererile ulterioare acesteia
 - dacă parolele nu corespund, utilizatorul primește pe ecran un mesaj corespunzător
- dacă nu există, este afișat un mesaj corespunzător

3. Logout: fiecare utilizator are pe pagina propriului profil un buton de logout care face ca token-ul să nu mai fie valid și șterge cookie-ul de sesiune

4. Resetare parolă

- utilizatorul accesează pagina `/forgotpassword` și își adaugă email-ul într-un formular
- dacă email-ul există este generat un token aleator care este stocat în câmpul `users.reset_token` și este accesat un URL de forma `/resetpassword/:token` (unde `token = users.reset_token`), această pagină conține un formular în care utilizatorul poate adăuga noua parolă ce va fi salvată și în baza de date în locul corespunzător
- dacă email-ul nu există este afișat un mesaj corespunzător

5. Gestionare sesiuni: menținerea autentificării între cereri se face prin utilizarea cookie-ului de sesiune ce salvează token-ul cu datele utilizatorului logat, iar la fiecare cerere este verificat payload-ul (`jwt.verify(req.cookies.auth_token, cheie_secreta)` este extras la fiecare rută ce presupune un user logat și conține câmpurile `userId` și `manager`)